SPARQL 1.2 Query Results JSON Format

W3C Working Draft 14 August 2025

▼ **More details about this document**

This version:

https://www.w3.org/TR/2025/WD-sparql12-results-json-20250814/

Latest published version:

https://www.w3.org/TR/sparql12-results-json/

Latest editor's draft:

https://w3c.github.io/sparql-results-json/spec/

History:

https://www.w3.org/standards/history/sparql12-results-json/

Commit history

Test suite:

https://w3c.github.io/rdf-tests/

Latest Recommendation:

https://www.w3.org/TR/2013/REC-sparql11-results-json-20130321

Editors:

Andy Seaborne

Ruben Taelman

Gregory Williams

Thomas Pellissier Tanon

Former editors:

Andy Seaborne

Kendall Grant Clark

Lee Feigenbaum

Elias Torres

Feedback:

GitHub w3c/sparql-results-json (pull requests, new issue, open issues)

public-rdf-star-wg@w3.org with subject line `[sparql12-results-json] … message topic …` (archives)

Copyright © 2012-2025 World Wide Web Consortium. W3C® liability, trademark and permissive document license rules apply.

Abstract

SPARQL is a set of standards for the query and update of RDF data, along with ways to access such data over the web. This document describes the representation of SELECT and ASK query results using JSON.

Status of This Document

This section describes the status of this document at the time of its publication. A list of current W3C publications and the latest revision of this technical report can be found in the W3C standards and drafts index.

This specification is published by the RDF Star Working Group as part of the update of specifications for format and errata.

This document was published by the RDF & SPARQL Working Group as a Working Draft using the Recommendation track.

Publication as a Working Draft does not imply endorsement by W3C and its Members.

This is a draft document and may be updated, replaced, or obsoleted by other documents at any time. It is inappropriate to cite this document as other than a work in progress. Future updates to this specification may incorporate new features.

This document was produced by a group operating under the W3C Patent Policy. W3C maintains a public list of any patent disclosures made in connection with the deliverables of the group; that page also includes instructions for disclosing a patent. An individual who has actual knowledge of a patent that the individual believes contains Essential Claim(s) must disclose the information in accordance with section 6 of the W3C Patent Policy.

This document is governed by the 03 November 2023 W3C Process Document.

§ Set of Documents

This document is one of twelve SPARQL 1.2 documents produced by the RDF & SPARQL Working Group.

▶ List of documents

Table of Contents

Abstract

Status of This Document
Set of Documents

1. Introduction
1.1 Version Announcement

2. JSON Results Object

3. Variable Binding Results
3.1 "head"
3.1.1 "vars"
3.1.2 "link"
3.1.3 "version"
3.2 "results"
3.2.1 "bindings"
3.2.2 Encoding RDF terms

4. Boolean Results
4.1 "head"
4.1.1 "link"
4.2 "boolean"

5. Examples

5.1 Variable Binding Results Examples
5.2 Variable Binding Results Examples with Triple Terms

6. Internet Media Type, File Extension and Macintosh File Type

A. Changes between SPARQL 1.1 Query Results JSON Format and SPARQL 1.2 Query Results JSON Format

B. Privacy Considerations

C. Security Considerations

D. Internationalization Considerations

E. Index
E.1 Terms defined by this specification
E.2 Terms defined by reference

F. References
F.1 Informative references

§ 1. Introduction

This document describes how to serialize SPARQL results (SELECT and ASK query forms) in a JSON format. The format is designed to be a complete representation of the information in the query results. The results of a SELECT query are serilialized as an array, where each array element is one "row" of the query results; the results of an ASK query give the boolean value of the query result.

An Internet Media Type is provided for `application/sparql-results+json`.

There is also a *SPARQL 1.2 Query Results XML Format* which follows a similar design pattern but uses XML as the serialization.

Unless otherwise noted in the section heading, all sections and appendices in this document are normative.

§ 1.1 Version Announcement

An optional version can be given to SPARQL results.

The `version` Media Type parameter *MAY* be set to `1.2` to indicate possible use of RDF 1.2 features in the SPARQL results. Alternatively, the `"version"` member *MAY* be used to specify this version.

Possible values for the version string are discussed in RDF 1.2 Concepts and Abstract Syntax.

When providing content over HTTP, servers can announce the version using the optional `version` Media Type parameter:

```
GET /sparql/?query=PREFIX%20dc%3A%20%3Chttp%3A%2F%2Fpurl.org%2Fdc
Host: www.example
Accept: application/sparql-results+json; version=1.2
```

> NOTE
>
> When using RDF 1.2-specific features, such as initial text direction, the specific RDF version can be announced using either the `version` Media Type parameter or the `"version"` member early in the document. This allows parsers that do not support these features to detect the presense of such features early, and potentially inform the user, giving them an opportunity to stop the job or otherwise act on the fact that some amount of the input data will not be processed as desired.

§ 2. JSON Results Object

The results of a SPARQL Query are serialized in JSON as a single top-level JSON object. This object has a `"head"` member and either a `"results"` member or a `"boolean"` member, depending on the query form.

This example shows the results of a `SELECT` query. The query solutions are represented in an array which is the value of the `"bindings"` key, in turn part of an object that is the value of the `"results"` key:

```
{
  "head": { "vars": [ "book" , "title" ]
  } ,
  "results": {
    "bindings": [
      {
        "book": { "type": "uri" , "value": "http://example.org/bc
        "title": { "type": "literal" , "value": "Harry Potter anc
      } ,
      {
        "book": { "type": "uri" , "value": "http://example.org/bc
        "title": { "type": "literal" , "value": "Harry Potter anc
      } ,
      {
        "book": { "type": "uri" , "value": "http://example.org/bc
        "title": { "type": "literal" , "value": "Harry Potter anc
      } ,
      {
        "book": { "type": "uri" , "value": "http://example.org/bc
        "title": { "type": "literal" , "value": "Harry Potter anc
      } ,
      {
        "book": { "type": "uri" , "value": "http://example.org/bc
        "title": { "type": "literal" , "value": "Harry Potter anc
      } ,
      {
        "book": { "type": "uri" , "value": "http://example.org/bc
        "title": { "type": "literal" , "value": "Harry Potter anc
      } ,
      {
        "book": { "type": "uri" , "value": "http://example.org/bc
        "title": { "type": "literal" , "value": "Harry Potter anc
      } ,
      {
        "book": { "type": "uri" , "value": "http://example.org/bc
        "title": { "type": "literal" , "value": "Harry Potter anc
      }
    ]
  }
}
```


This example shows the result from an ASK query:

```
{
  "head" : { } ,
  "boolean" : true
}
```

Other keys, with different names, may be present in the JSON Results Object but are not defined by this specification.

§ 3. Variable Binding Results

The results of a SPARQL `SELECT` query are serialized as an array of bindings of variables. The value of the `"head"` key is an array of all variables projected in the query's `SELECT` clause.

§ 3.1 "head"

The `"head"` member gives the variables mentioned in the results, may contain a `"link"` member, and may contain a `"version"` member.

```
{
"head" {
   "vars" : [ ... ] ,
   "link" : [ ... ] ,
   "version" : "..." }
```

§ 3.1.1 "vars"

The `"vars"` member is an array giving the names of the variables used in the results. These are the projected variables from the query. A variable is not necessarily given a value in every query solution of the results.

```
"vars" : [ "book" , "title" ]
```

The order of variable names should correspond to the variables in the SELECT clause of the query, unless the query is of the form `SELECT *` in which case order is not significant.

§ 3.1.2 "link"

The optional `"link"` member gives an array of URIs, as strings, to refer for further information. The format and content of these link references is not defined by this document.

```
"link" : [ "http://example/dataset/metadata.ttl" ]
```

§ 3.1.3 "version"

The optional `"version"` string member, along with an optional `version` Media Type parameter, *MAY* be used to announce that RDF 1.2 functionalities are used in the results.

```
"version" : "1.2"
```

§ 3.2 "results"

The value of the `"results"` member is an object with a single key, `"bindings"`.

§ 3.2.1 "bindings"

The value of the `"bindings"` member is an array with zero or more elements, one element per query solution. Each query solution is a JSON object. Each key of this object is a variable name from the query solution. The value for a given variable name is a JSON object that encodes the variable's bound value, an RDF term. There are zero elements in the array if the query returned an empty solution sequence. Variables names do not include the initial "?" or "$" character. Each variable name that appears as a key within the `"bindings"` array will have appeared in the `"vars"` array in the results header.

A variable does not appear in an array element if it is not bound in that particular query solution.

The order of elements in the bindings array reflects the order, if any, of the query solution sequence.

```
"bindings" : [
  {
    "a" : { ... } ,
    "b" : { ... }
  } ,
  {
    "a" : { ... } ,
    "b" : { ... }
  }
]
```

If the query returns no solutions, an empty array is used.

```
"bindings" : []
```

§ 3.2.2 Encoding RDF terms

An RDF term (IRI, literal, blank node, or triple term) is encoded as a JSON object. All aspects of the RDF term are represented. The JSON object has a `"type"` member and other members depending on the specific kind of RDF term.

RDF Term	JSON form
IRI ***I***	`{"type": "uri", "value": "I"}`
Literal ***S***	`{"type": "literal","value": "S"}`
Literal ***S*** with language tag ***L*** without base direction	`{ "type": "literal", "value": "S", "xml:lang": "L"}`
Literal ***S*** with language tag ***L*** with base direction ***L***	`{ "type": "literal", "value": "S", "xml:lang": "L", "its:dir": "B"}`
Literal ***S*** with datatype IRI ***D***	`{ "type": "literal", "value": "S", "datatype": "D"}`
Blank node, label ***B***	`{"type": "bnode", "value": "B"}`
Triple term, with subject ***S***, predicate ***P***, and object ***O***	`{"type": "triple", "value": {"subject": "S", "predicate": "P", "object": "O"}}`

The blank node label is scoped to the results object. That is, two blank nodes with the same label in a single SPARQL Results JSON object are the same blank node. This is not an indication of any internal system identifier the SPARQL processor may use. Use of the same label in another SPARQL Results JSON object does not imply it is the same blank node.

The subject, predicate, and object of a triple term are encoded using the same format, recursively.

> NOTE
>
> The `xml:lang` and `its:dir` keys resemble terms in XML namespaces due to alignment with the *SPARQL 1.2 Query Results XML Format* specification, but such namespaces do not exist in JSON, which means that these keys are to be used "as is".

§ 4. Boolean Results

The results of a SPARQL `ASK` query are serialized as a boolean value, giving the result of the query evaluation.

§ 4.1 "head"

§ 4.1.1 "link"

The `"link"` member has the same format as the SELECT `"link"` member.

§ 4.2 "boolean"

The result of an `ASK` query form are encoded by the `"boolean"` member, which takes either the JSON value `true` or the JSON value `false`.

```
"boolean" : true
```

§ 5. Examples

This section is not normative.

§ 5.1 Variable Binding Results Examples

The following JSON is a serialization of the XML document output.srx:

```
{
  "head": {
    "link": [ "http://www.w3.org/TR/rdf-sparql-XMLres/example.rq'
    "vars": [
      "x",
      "hpage",
      "name",
      "mbox",
      "age",
      "blurb",
      "friend"
    ]
  },
  "results": {
    "bindings": [
      {
        "x":     { "type": "bnode",   "value": "r1" },
        "hpage": { "type": "uri",     "value": "http://work.examp
        "name":  { "type": "literal", "value": "Alice" },
        "mbox":  { "type": "literal", "value": "" },
        "blurb": {
          "datatype": "http://www.w3.org/1999/02/22-rdf-syntax-ns
          "type": "literal",
          "value": "<p xmlns=\"http://www.w3.org/1999/xhtml\">My
        },
        "friend": { "type": "bnode", "value": "r2" }
      },
      {
        "x":      { "type": "bnode",   "value": "r2" },
        "hpage":  { "type": "uri",     "value": "http://work.exam
        "name":   { "type": "literal", "value": "Bob", "xml:lang'
        "mbox":   { "type": "uri",     "value": "mailto:bob@work.
        "friend": { "type": "bnode",   "value": "r1" }
      }
    ]
  }
}
```

§ 5.2 Variable Binding Results Examples with Triple Terms

The following JSON is a serialization of the XML document output-triple-terms.srx that contains triple terms:

```
{
  "head": {
    "link": [ "http://www.w3.org/TR/rdf-sparql-XMLres/example-tri
    "vars": [
      "x",
      "name",
      "triple"
    ]
  },
  "results": {
    "bindings": [
      {
        "x":    { "type": "bnode",   "value": "r1" },
        "name": { "type": "literal", "value": "Alice" },
        "triple": {
          "type": "triple",
          "value": {
            "subject":   { "type": "uri",     "value": "http://ex
            "predicate": { "type": "uri",     "value": "http://ex
            "object":    { "type": "literal", "value": "Alice", "
          }
        }
      },
      {
        "x":    { "type": "bnode",   "value": "r2" },
        "name": { "type": "literal", "value": "Bob", "xml:lang":
        "triple": {
          "type": "triple",
          "value": {
            "subject":   { "type": "uri",     "value": "http://ex
            "predicate": { "type": "uri",     "value": "http://ex
            "object":    { "type": "literal", "value": "Bob", "da
          }
        }
      }
    ]
  }
}
```

§ 6. Internet Media Type, File Extension and Macintosh File Type

The Internet Media Type (formerly known as MIME Type) for the SPARQL Query Results JSON Format is "application/sparql-results+json".

It is recommended that SPARQL Query Results JSON Format files have the extension ".srj" (all lowercase) on all platforms.

It is recommended that SPARQL Query Results JSON Format files stored on Macintosh HFS file systems be given a file type of "TEXT".

This information that follows is intended to be submitted to the IESG for review, approval, and registration with IANA.

Type name:
application

Subtype name:
sparql-results+json

Required parameters:
None

Optional parameters:
None

Encoding considerations:
The encoding considerations of the SPARQL Query Results JSON Format is identical to those of the "application/json" as specified in [*RFC4627*].

Security considerations:
SPARQL query results uses URIs. See Section 7 of [*RFC3986*].
SPARQL query results uses IRIs. See Section 8 of [*RFC3987*].
The security considerations of the SPARQL Query Results JSON Format is identical to those of the "application/json" as specified in [*RFC4627*].

Interoperability considerations:
There are no known interoperability issues.

Published specification:
http://www.w3.org/TR/sparql11-results-json/

Applications which use this media type:

No known applications currently use this media type.

Additional information:
Magic number(s):

n/a

File extension(s):

".srj"

Macintosh file type code(s):

"TEXT"

Person & email address to contact for further information:

Andy Seaborne <public-rdf-dawg-comments@w3.org>

Intended usage:

COMMON

Restrictions on usage:

None

Author/Change controller:

The SPARQL specification is a work product of the World Wide Web Consortium's SPARQL Working Group. The W3C has change control over these specifications.

A. Changes between SPARQL 1.1 Query Results JSON Format and SPARQL 1.2 Query Results JSON Format

This section is non-normative.

- Fix link to incorrect srx file in 5. Examples
- Add JSON Schema file for SPARQL Query Results JSON
- Use Media Type instead of MIME Type in 6. Internet Media Type, File Extension and Macintosh File Type
- Allow triple terms to be expressed in 3.2.2 Encoding RDF terms
- Support directional language-tagged strings in 3.2.2 Encoding RDF terms
- Add version announcement in 1.1 Version Announcement and 3.1.3 "version"

B. Privacy Considerations

TODO

C. Security Considerations

TODO

D. Internationalization Considerations

TODO

E. Index

E.1 Terms defined by this specification

E.2 Terms defined by reference

[*RDF12-CONCEPTS*] defines the following:

initial text direction

RDF 1.2 Concepts and Abstract Syntax

[*RFC4627*] defines the following:

JSON

[*SPARQL12-CONCEPTS*] defines the following:

SPARQL

§ F. References

§ F.1 Informative references

[RDF12-CONCEPTS]

RDF 1.2 Concepts and Abstract Syntax. Olaf Hartig; Pierre-Antoine Champin; Gregg Kellogg; Andy Seaborne. W3C. 7 August 2025. W3C Working Draft. URL: https://www.w3.org/TR/rdf12-concepts/

[RFC3986]

Uniform Resource Identifier (URI): Generic Syntax. T. Berners-Lee; R. Fielding; L. Masinter. IETF. January 2005. Internet Standard. URL: https://www.rfc-editor.org/rfc/rfc3986

[RFC3987]

Internationalized Resource Identifiers (IRIs). M. Duerst; M. Suignard. IETF. January 2005. Proposed Standard. URL: https://www.rfc-editor.org/rfc/rfc3987

[RFC4627]

The application/json Media Type for JavaScript Object Notation (JSON). D. Crockford. IETF. July 2006. Informational. URL: https://www.rfc-editor.org/rfc/rfc4627

[SPARQL12-CONCEPTS]

SPARQL 1.2 Concepts. The W3C RDF & SPARQL Working Group. W3C. W3C Editor's Draft. URL: https://w3c.github.io/sparql-concepts/spec/

[SPARQL12-ENTAILMENT]

SPARQL 1.2 Entailment Regimes. Peter Patel-Schneider. W3C. 19 December 2024. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-entailment/

[SPARQL12-FEDERATED-QUERY]

SPARQL 1.2 Federated Query. Ruben Taelman; Gregory Williams. W3C. 5 May 2025. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-federated-query/

[SPARQL12-GRAPH-STORE-PROTOCOL]

SPARQL 1.2 Graph Store Protocol. Andy Seaborne; Thomas Pellissier Tanon. W3C. 19 December 2024. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-graph-store-protocol/

[SPARQL12-NEW]

What's New in SPARQL 1.2. The W3C RDF & SPARQL Working Group. W3C. W3C Editor's Draft. URL: https://w3c.github.io/sparql-new/spec/

[SPARQL12-PROTOCOL]

SPARQL 1.2 Protocol. Andy Seaborne; Ruben Taelman; Gregory Williams; Thomas Pellissier Tanon. W3C. 24 June 2025. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-protocol/

[SPARQL12-QUERY]

SPARQL 1.2 Query Language. Olaf Hartig; Andy Seaborne; Ruben Taelman; Gregory Williams; Thomas Pellissier Tanon. W3C. 11 August 2025. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-query/

[SPARQL12-RESULTS-CSV-TSV]

SPARQL 1.2 Query Results CSV and TSV Formats. Ruben Taelman; Gregory Williams; Thomas Pellissier Tanon. W3C. 19 December 2024. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-results-csv-tsv/

[SPARQL12-RESULTS-XML]

SPARQL 1.2 Query Results XML Format. Ruben Taelman; Dominik Tomaszuk; Thomas Pellissier Tanon. W3C. 27 December 2024. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-results-xml/

[SPARQL12-SERVICE-DESCRIPTION]

SPARQL 1.2 Service Description. Ruben Taelman; Gregory Williams. W3C. 19 December 2024. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-service-description/

[SPARQL12-UPDATE]

SPARQL 1.2 Update. Ruben Taelman; Andy Seaborne; Thomas Pellissier Tanon. W3C. 6 February 2025. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-update/

↥